Introduction to the Unix Shell

Laura Prieto Saavedra
Scientist and Research Software Engineer
This work is licensed under a CC BY 4.0 license.

February 05, 2026, Software and Data Seminar

Context

Why is this relevant for you?

- Do you use internal computers or other clusters? → basic interaction
- Do you deal with large data files or chunks of data? → move, copy, backup
- Do you post-process results? → naming, extraction, formats
- Do you want to avoid human errors when handling data?
- Do you want to work efficiently?

Goal: to learn basic commands of the Unix shell so that we feel confident when using the terminal for our work.

How? I will present the concepts and commands, and you will be able to try them out at the same time. This will be an **interactive seminar**.

Note: the lesson is largely based on the Software Carpentry lesson about the UNIX shell (see references at the end).

Prerequisites: what do you need?

A laptop with a `bash` terminal:

- For most Linux distributions: `Ctrl+t`.
- For macOS: open terminal and type `bash`.
- For Windows users an installation guide is presented here: `https://linuxconfig.org/ubuntu-22-04-on-wsl-windows-subsystem-for-linux`.

Download the sample directory and place it on your Desktop:

- Posted in the Software and Data Seminar chat on Nextcloud.
- Or clone the WIAS GitLab repo.

During the presentation:

- Warnings will be shown in red.
- Best practices will be shown in orange.
- Tasks will be shown in blue.

Content

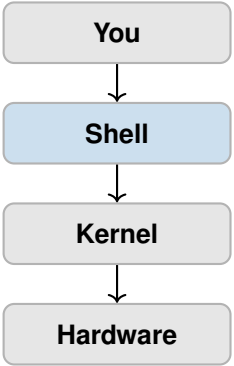
1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Introduction: the Unix operating system

- The UNIX operating system started at Bell Laboratories in 1969.
- In 1974, it was licensed to universities and then it spread world-wide.
- UNIX, also known as kernel, manages memory, processes, files, etc.
- The Unix shell is a program that allows you to talk to the UNIX operating system.



How does it work?

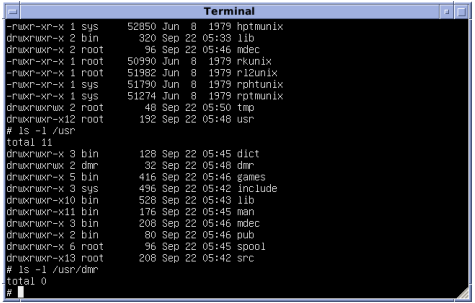
Introduction: the Unix shell

How does it work?

- It reads commands you type.
- It interprets them.
- It asks the OS kernel to run programs.

There are different Unix shells:

- `sh` (Bourne shell): the original
- `bash` (Bourne Again Shell): the most common
- `zsh`
- `ksh`
- `fish`



Version 7. Taken from [Huihermit, 2025].

But why do we need a shell, a.k.a. Command Line Interface (CLI) if we have a Graphical User Interface (GUI)?

Introduction: GUI vs CLI

Graphical User Interface (GUI)

Pros:

- ✓ Intuitive and visual.
- ✓ Beginner friendly.
- ✓ Good for interactive tasks.
- ✓ Easy to learn.

Cons:

- × Slow for repetitive tasks.
- × Hard to automate.
- × Error prone.
- × Hides what is actually happening.

Command Line Interface (CLI)

Pros:

- ✓ Fast.
- ✓ Reproducible (scripts).
- ✓ Offers precise control.
- ✓ Excellent for remote connections.

Cons:

- × Takes time to learn.
- × Not visually intuitive.
- × Easy to mess up if you do not know what you are doing.

This seminar should help you to learn all the basics. Let us focus on the `bash` terminal.

Setting up your bash terminal

1. Open a new prompt, this is what you will see:

```
username@computer_name:~$
```

2. Check whether you are at the home directory using the :

```
username@computer_name:~$ pwd
```

   This should give the current path: `/home/username`. If not type `cd`. In this presentation, commands will be shown without the username and computer name.

New commands will be introduced as follows:

Command 0: display a line of text

```
~$ echo word
```

Task: test the command with a word and phrase.

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

File system and current path

- The part of the operating system responsible for managing files and directories is called the **file system**.
- It organizes data into files, which hold information, and directories (a.k.a. folders) which hold files or other directories.

Command 1: print working directory

```
~$ pwd
```

Output:

```
/home/username
```

- This is the current user's home directory, which is also "~" in the shell.
- The topmost directory is the **root directory**: the leading slash "/" in the `pwd` output. Several directories are stored there: `bin`, `home`, `tmp`, etc.
- Our current directory is stored inside `home`.

Task: check your current path. Does it have the same structure?

Listing the contents of the current directory

Command 2: listing

```
~$ ls
```

Output:

```
Desktop script file
```

- It prints the names of all files and directories.
- Colors change depending on the settings of your terminal.
- One can also add options or flags for additional information, example: `ls -F` or `ls --classify` to classify the files depending on their type.

Each command may have lots of options, how do we learn about them?

```
~$ ls --help
```

```
~$ man ls
```

Task: use the command and add flags `-l`, `-h`, `-r`, `-t`. What do they do? Combine `-l` and `-h`, what do you see? Does it give the same as `-lh`?

Listing the contents of other directories and change location

One can also use those commands to list the contents of a different directory, for example to list all files in Desktop, execute:

```
~$ ls -F Desktop  OR ls --classify Desktop
```

If the directory does not exist you will simply get an error.

Best practices: use short commands in the shell to type faster, use long commands in scripts to provide clarity.

Command 3: change directory

```
~$ cd Desktop
```

Output: None.

- It moves us from the home directory to the Desktop directory.
- Task: try it out and verify the folder using `pwd` and its contents using `ll`.

Now we know how to go into a subdirectory, but how do we go back?

See directories above the current location

To move up one directory level:

```
~$ cd ..
```

where `..` is an argument that means special directory, i.e., "the directory containing this one", a.k.a. the parent directory. Task: use the command and `pwd` to verify it. Move around different directories.

Note: this directory does not appear when using `ls`. But if we add the `-a` option, you will see it. Along with `.` which is the current working directory.

Task:

- What happens if you only type `cd`?
- What happens if you type `cd -`?

These three basic commands (along with their options and arguments) allow you to navigate the filesystem on your computer, but what else can we do?

Working with files and directories

Command 4: make a directory

```
~$ mkdir project
```

Output: None.

- `project` is a **relative path**, it does not have a leading slash.
- Task: use it and verify it was created using `ll`.

Other options:

- You can create more than one directory at the time:

```
~$ mkdir -p ./project/data ./project/results
```

- You can print all the subdirectories using `ls -FR ./project`.

Best practices: for names of files and directories do not use spaces, do not begin with dash (-), stick to lowercase letters, numbers, period, dash and underscore(_). If spaces are absolutely necessary use "`project 1`".

Creating a text file

You can create a text file using your favorite editor (`vim`, `nano`, etc), but you can also create in the shell using the command `touch`.

Command 5: change filestamps

```
~$ touch file_1.txt
```

Output: None.

- It creates a file of size 0 bytes, i.e., it is empty. Useful when our program requires that a file exists.
- Task: use it and verify it was created using `ls -l`.

Best practices: one could choose any extension for the file, it is up to us to choose the filename extension according to our needs (`csv`, `txt`, etc).

What can we do with these files and directories?

Moving and copying files and directories

Command 6: moving

```
~$ mv filename location
```

Output: None.

- If the location is in another directory, it will move it there.
- If the location is in the same directory, it will rename the file.

Command 7: copying

```
~$ cp filename location
~$ cp -r dirname location
```

Output: None.

- It is important to include the `-r` flag for a directory.

Warning: if the file exists in any case, it will overwrite it. Best practices: use flag `-i` to avoid data loss.

Task: create a folder using `mkdir`. Create several files within it using `touch`. Move them and copy them around using these commands.

Deleting files and directories

Command 8: deleting

```
~$ rm filename
~$ rm -r dirname
```

Output: None.

- Warning: deleting is forever! Best practices: use the `-i` flag to confirm deletion.
- It is important to include the `-r` flag for a directory.
- Task: remove a file and a directory.

All these commands work for one file. What if I have multiple files?

Wildcards: using `*` represents zero or more characters. If we have several files named `file_1.txt`, `file_2.txt`, etc, we can move all of them to a directory as follows: `mv file*.txt dirname`. These wildcards can be used with all commands. Task: Test this example using wildcards.

Nice... so I can do basically the same as in the GUI with some commands? Let's learn some powerful features of the shell!

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Capturing output from commands

Go to the following folder `cd ~/Desktop/unix_shell_exercises/alkanes/`.

Command 9: word count

```
$ wc file
```

Output:

```
lines words characters file
```

- To show only the number of lines use the flag `-l`, the number of words use the flag `-w`, and the number of characters use the flag `-m`.

Command 10: redirect

```
$ wc file > counts.txt
```

Output: None.

- Redirects the output to a file. Warning: If the file exists, it will be overwritten.
- If you rather append the contents to a file use >>.

Task: test word count with all files in the folder (use wildcards `*.pdb`) and redirect the output to a file called "`counts.txt`".

Printing the content of the file

Command 11: concatenate

```
~$ cat file(s)
```

Output: content of file.

- It allows to send the content of the file to the screen.
- If used with several files, it will print the contents of files, one after another.
- Useful but not convenient for large files. Best practices: use `clear` to clean your shell.

Command 12: show less

```
~$ less large_file
```

Output: interactive screen.

- This displays only a screenful of the file and stops.
- You can move forward by pressing the spacebar and back by pressing `b`.
- Press `q` to quit.

Task: test the first option with the "`counts.txt`" file. Then use the `cat` command to concatenate all `.pdb` files in a new `.txt` file and test `less`.

Filtering output

Command 13: alphanumerical sorting

```
~$ sort file
```

Output: a sorted list.

Command 14,15: print some lines

```
~$ head -n 1 file
~$ tail -n 1 file
```

Output: specified lines.

- Numerical sort with flag `-n`.
- Its output can be redirected using > to another file. Warning: do not try to redirect it to the same file.
- Task: use it with the `numbers.txt` file and save output in `sorted.txt`. What happens if you use it with `letters.txt`?
- It prints lines from the top or the bottom of the file.
- With the `-n` flag we can specify any number of lines.
- Task: test the command with the `sorted.txt` file.

Passing output to another command

Until now, we have executed one command at a time. We sorted the numbers and saved it to a file. Then, we printed the first line of the new file. We can do all of this in one line:

Command 16: passing output to another command

```
~$ sort -n numbers.txt | head -n 1
```

Output:

```
2
```

- The vertical bar is called a **pipe**, it tells the shell that we want to use the output of the command on the left as input to the command on the right.
- This completely removes the need of saving the output in an additional file. Task: test it.

This can be done for more than two commands!

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Perform same action on many files

We have been applying all commands to a single file. What if I want to perform the same action on many files?
We use **loops**. What is the syntax in `bash`?

```
# The for indicates the start of for-loop command
for thing in list_of_things
# The word do indicates the start of job execution list
do
    # Indentation not required, but helps for legibility
    command $thing
# The word done indicates the end of a loop
done
```

This should be familiar if you use any other programming language...

Note: There are also other flow control structures (if, while, functions) that will not be covered in this introduction.

A concrete example of a loop

Let us look at an example:

```
~$ for filename in file_1 file_2 file_3
> do
>    echo $filename
>    head -n 2 $filename | tail -n 1
> done
```

- The shell prompt changes from `$` to `>` and back again as we type our loop in the shell.
- You can use a semicolon `;` to separate two commands and write only one line.
- Inside the loop we call each of the variables by using `$` at the beginning.
- It is also common to put the variable names into curly braces: `${filename}`.
- We could have chosen instead of filename any other word. Best practices: choose meaningful names and use curly braces.

Task: Print the name of each of the files in the exercises directory using a loop. Use the same syntax but with numbers instead of files to create 10 files named `file_number.txt`.

Some cases where loops are useful

- To loop over thousands of files with the same extension using the wildcard and concatenating the output to one file:

```
~$ for file in *.dat; do echo $file; cat $file >> all.txt; done
```

Task: test it with all the files in the alkanes directory.

- To back-up the same thousand files. But can we do this already with the copy command?

```
~$ cp *.dat original-*.dat
```

No, it would throw an error since it has more than two inputs. Instead we can use a loop:

```
~$ for filename in *.dat; do cp $filename original-$filename; done
```

Task: test it by making a back-up of all the `*.pdb` files in the alkanes directory.

Best practices: it is common to use the `echo` command as debugging technique.

The commands are getting longer...

Some additional tips for the usage of the shell:

- To see previous commands simply use the ↑.
- To move to the beginning of a line type `Ctrl`+A or the `Home` key.
- To move to the end of a line type `Ctrl`+E or the `End` key.
- Use the history command:

Command 17: list previously executed commands

```
~$ history
```

Output: last few hundred lines

- More useful if used along with `tail -n 5`.
- Check the line of the command and repeat it by typing `!line_number`.
- Task: try all the options.

- `Ctrl`+R enters a history search mode and finds the most recent command that matches the text you enter. Press it several times for earlier matches. Then use left and right arrows to modify it.

Nested loops

Nested loops are also supported. For example, we can use them to create certain directory structure to organize some experiments:

```
~$ for mesh in small medium large
> do
>    for velocity in 20 40 60
>    do
>       mkdir $mesh-$velocity
>    done
> done
```

Task: test it. What happens if I change the dash to an underscore?

This is nice but slow if I am doing repetitive tasks... for example, post-processing some data. Is there a way of making this faster?

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Shell scripts

We take the commands that we repeat frequently and save them in files so that we can re-run all those operations again later by typing a single command.

These shell scripts are **small programs** that will:

- Make your work faster.
- Avoid that you have to rewrite over and over the same commands.
- Make it more accurate → fewer chances for typos.
- Make it easier for someone that takes over your project (or even yourself after a while) to reproduce the results only by running a script.

How do they work? you write the commands on a file using your favorite editor. Then the script is executed in one line.

Best practices: it is common to save the file with a `.sh` extension for scripts meant to be read/edited.

A concrete example

Suppose we want to create a script that sorts the numbers file and prints only the first line by appending it to a file called results. We use an editor, for example, `nano`:

```
~$ nano my_script.sh
```

and copy and save the following line:

```
sort -n numbers.txt | head -n 1 >> results.txt
```

Since we are using the `bash` shell, we can execute the command as follows:

```
~$ bash my_script.sh
```

How does it work? You explicitly start the program bash, it reads the input file and executes the commands.

What if I want to execute the script for a different file? Do I have to open the script and change it every time?

Passing arguments to a script

Replace the filename by a special variable called $1:

```
sort -n $1 | head -n 1 >> results.txt
```

Now we can simply execute the script as follows:

```
~$ bash my_script.sh other_file.txt
```

We can use as many additional arguments as we want, for example, we can add an argument for the number of printed lines:

```
sort -n $1 | head -n $2 >> results.txt
```

Best practices: add comments at the top of your script so that anyone can understand it.

```
# Select a certain amount of lines when sorting file.
# Usage: bash my_script.sh filename num_lines
sort -n $1 | head -n $2 >> results.txt
```

Task: create the script and test it with `numbers.txt` and `letters.txt`.

Other arguments of a script

What if I want to process hundred files located in different directories in a single pipeline? We can sort data files by length of a directory in the shell terminal as follows:

```
~$ wc -l *.dat | sort -n
```

We can add it as it is in a script, but it would only take into account the `.dat` files in the directory where the script is executed. Instead we can use the variable `$@`, which means "all the command arguments to the shell script":

```
# Sort files by their length in lines.
# Usage: bash sorting.sh *.dat ../another_dir/*.dat
wc -l $@ | sort -n
```

Best practices: develop scripts by running commands in the shell prompt and testing them a few times before saving them in a file for reuse.

Another way of running scripts...

One can also run a script as an executable:

```
~$ ./sorting.sh
```

However:

- It needs to have the right permissions, which can be adjusted using the command `chmod +x file`.
- It needs a “shebang”, i.e., a first line that tells the kernel what program to use to run the script:

  ```
  #!/usr/bin/env bash
  ```

  to ensure that your script is portable (runs also on non-bash terminals) and runs as expected.

Best practices: always use a shebang, even if you usually run your script in bash terminals.

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Finding things in the shell using `grep`

To select lines from text files that match single patterns:

Command 18: global/regular expression/print

```
~$ grep pattern file(s)
```

Output: all lines containing that pattern.

- It is by default case-sensitive.
- The pattern does not have to be a complete word. If we are searching for complete words we need the `-w` flag.
- If the pattern is a phrase put the phrase in quotes.

Other useful flags for this command:

- `-n` shows the line number of the match.
- `-i` makes the search case-insensitive.
- `-v` inverts the search, i.e. outputs the lines that do not contain the pattern.

Task: Test it with the `wias/mission.txt` file. How many times is WIAS and research mentioned? Experiment with the flags.

Finding things in the shell using `grep`

- We can search recursively within a directory:

```
~$ grep -r "seen" .
```

Task: repeat the same exercise with all the files in the `wias` folder.

Why is it so powerful? Patterns can include wildcards, the technical name for this is "regular expressions". As an example, to find the lines that have an 'o' in the second position we can use:

```
~$ grep -E "^.o" file.txt
```

where `-E` means an extended regular expression and prevents the program from interpreting it.

Note: regular expressions are a whole thing on their own. If you want to learn more about it, I recommend the Library Carpentry lesson at `https://librarycarpentry.github.io/lc-data-intro/01-regular-expressions.html`.

Finding things in the shell using `find`

While `grep` allows to find lines in files, another command allows you to find files themselves:

Command 19:

```
~$ find location flags
```

Output: list with all finds within that directory according to flag

- `find . -type d`: finds all directories.
- `find . -type f`: finds all files.
- `find . -name "*.txt"`: finds all files that match name.

Task: go back to the `unix_shell_exercises` folder and test the find command for folders and files.

This is simple but powerful when you combine it with other tools!

Combining `find` with other commands

We can use find to list certain files and then use them as arguments for other commands:

```
~$ wc -l $(find . -name "*.txt")
```

The output would be something like:

```
  32 ./wias/machine_learning.txt
 129 ./wias/history.txt
 161 total
```

The shell first executes whatever is inside `$()`, then replaces it with the command output. Another common combination is with `grep`:

```
~$ grep "software" $(find . -name "*.txt")
```

It first find the files that match a pattern and then it looks for lines that match another pattern within those files. Task: test it within the exercise folder.

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Other commands: `cut` and `uniq`

Command 20: to cut out something

```
~$ cut -f 2 file.csv
```

Output: prints the requested content.

- By default it expects the information to be delimited into columns by a `tab` character.
- Use flag `-d` to specify other delimeters, i.e., `-d ,`.
- Use flag `-f` to specify that we want to extract the second field (column).

Command 21: report or omit repeated lines

```
~$ uniq file.txt
```

Output: prints everything but repeated lines

- Use flag `-c` to print the lines plus the number of entries for each line.

Other commands: `sed` and `awk`

- `sed` is a program build for **editing** files:

Command 22: using `sed` for substitution

```
~$ sed 's/pattern/replacement
    /g' file
```

Output: prints the modified content.

- s means substitution
- Replaces all instances of a pattern with replacement in file.
- With the `-i` flag the substitutions happen inline.

- `awk` is a program build for **parsing** files:

Command 23: using `awk` to print something

```
~$ awk '{print $2}' file$}
```

Output: prints the requested content.

- Prints the second whitespaced-separated column of every line of the file.
- Add flag `-F,` for comma separated info.

Archiving and compression

Command 24: creates, extracts and manipulate files

```
~$ tar -xf folder.tar file
```

- To create use flag `-c` instead of `-x`.
- Use flag `-tf` to list the contents of an archive without extracting them.
- Use `-v` to print files to terminal.

Command 25: compression utility

```
~$ zip compressed.zip file
```

- Use flag `-r` for folders instead of files.
- Use flag `-q` to not print the name of the compressed files.

Command 26: decompression utility

```
~$ unzip file
```

- Use flag `-d` for a destination folder.
- Use flag `-q` to not print the name of the extracted files.

Your `bashrc` file

- Short for `bash read command`. It is a configuration file for the Bash shell environment.
- Every time you open a new shell, this script is executed.
- It can be modified with any editor.

There you can:

- Define aliases, a.k.a. command shortcuts:

```
alias ll='ls -lah'
```

- Customize your prompt.
- Configure command history.
- Set environment variables:

```
export EDITOR=vim
export BROWSER=firefox
```

- ...

The environment variable `PATH`

A very important environment variable. It is a colon-separated list of directories that the shell searchers **in order** when you type a command without a full path.

```
~$ echo $PATH
```

Output: something similar to

```
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

when you type `ls`, `bash` does the following:

1. Looks for an executable with that name.
2. Checks each directory in `$PATH` from left to right.
3. Runs the first match it finds.

Without it you would have to type the absolute path of each program. To add something to your path you append it to the original one:

```
~$   export PATH="$NEW_PATH:$PATH"
```

How to deal with files in the terminal?

We already saw the basic options. Some additional options that people like to use:

- For text files or code:
  - Some terminal-based editors: `nano`, `vim`, `vi`.
  - Some GUI editors: `code` for VSCode, `subl` for Sublime, `gedit` for a very basic editor.
- For images or PDFs:
  - Use the system default's image/PDF viewer. Linux: `xdg-open`. macOS: `open`.
  - Install external tools for images, e.g., `feh`.
- For html pages:
  - Any program that is in `PATH`, e.g., `firefox`.

Content

1. Introduction
2. Files and directories
3. Pipes and filters
4. Loops
5. Shell scripts
6. Finding things
7. Other topics
8. Final remarks

Final remarks

- The Unix shell is old but has survived several decades because it is **one of the most productive programming environments** ever created.
- It's syntax might be difficult at first, but once you get used to it you can **master it** and use it to **automate your work**.
- GUIs might be easier to use at first, but its **speed** cannot be compared with the one of the shell.
- With these basic commands, you should be able to do **all the basics** in your own laptop, in the compute servers and in any other cluster.
- **Do not be afraid to experiment! There are tons of commands and options!**

Thank you for your attention!

Software and Data Seminar

References

Anish, Jose, and Jon.
The Missing Semester of Your CS Education.
https://missing.csail.mit.edu/.
[Accessed 02-02-2026].

Devenyi, Gabriel and Capes, Gerard and Morris, Colin and others (2019).
swcarpentry/shell-novice: Software carpentry: the unix shell.

Huihermit, title = Version 7 Unix, the Research Unix ancestor of all modern Unix systems, h. . h. (2025).
[Accessed 02-02-2026].

Kerniga, B. W. and Pike, R. (1984).
UNIX Programming Environment.
Bell Laboratories; Prentice-Hall.